Week 5 Day 1
Agent Foundations Write-up

This project implements a minimal AI agent from scratch in Python using an OpenAI-compatible API. The agent follows the ReAct (Reason → Act → Observe) pattern without using frameworks such as LangChain or LangGraph.

ReAct Loop

The agent begins by sending the user's request to the language model. If the model decides a tool is needed, it returns a tool call. The Python program executes the requested tool, appends the tool result to the conversation history, and sends the updated conversation back to the model. This process repeats until the model returns a final response or the maximum iteration limit is reached, preventing infinite loops.

Tool Schemas Used

Calculator: Performs add, subtract, multiply, and divide operations using the inputs *operation*, *a*, and *b*.

Weather Lookup: Returns fake weather information for a requested city from a small predefined weather dataset. This stub tool was used to demonstrate tool calling without relying on an external weather API.

Failure Modes Observed

1. **Ambiguous request:** When asked 'Is it hot today?', the agent requested the city instead of guessing, preventing incorrect tool usage.
2. **Tool error:** When a city was not available in the weather dataset, the tool returned an error instead of fabricating data.
3. **Missing capability:** When asked to send an email, the agent drafted the email instead of actually sending it because no email tool was available.

Conclusion

Building the agent manually demonstrated how conversation history, tool execution, memory, and iteration work together. It also showed why frameworks such as LangChain, LangGraph, and CrewAI are useful: they automate these repetitive tasks and make larger agent systems easier to build, debug, and maintain.